arXiv:2511.00184v1 [cs.DS] 31 Oct 2025

Scheduling Problems with Constrained Rejections

Sami Davies* Venkatesan Guruswami† Xuandi Ren‡

November 4, 2025

Abstract

We study bicriteria versions of Makespan Minimization on Unrelated Machines and Santa Claus by allowing a constrained number of rejections. Given an instance of Makespan Minimization on Unrelated Machines where the optimal makespan for scheduling n jobs on m unrelated machines is T, (Feige and Vondrák, 2006) gave an algorithm that schedules a $(1-1/e+10^{-180})$ fraction of jobs in time T. We show the ratio can be improved to $0.6533 > 1-1/e+0.02$ if we allow makespan $3T/2$. To the best our knowledge, this is the first result examining the tradeoff between makespan and the fraction of scheduled jobs when the makespan is not T or $2T$.

For the Santa Claus problem (the "Max-Min" version of Makespan Minimization), the analogous bicriteria objective was studied by (Golovin, 2005), who gave an algorithm providing an allocation so a $(1-1/k)$ fraction of agents receive value at least T/k, for any $k \in \mathbb{Z}^+$ and T being the optimal minimum value every agent can receive. We provide the first hardness result by showing there are constants $\delta, \varepsilon > 0$ such that it is NP-hard to find an allocation where a $(1-\delta)$ fraction of agents receive value at least $(1-\varepsilon)T$. To prove this hardness result, we introduce a bicriteria version of Set Packing, which may be of independent interest, and prove some algorithmic and hardness results for it. Overall, we believe these bicriteria scheduling problems warrant further study as they provide an interesting lens to understand how robust the difficulty of the original optimization goal might be.

1 Introduction

Two of the most central problems in scheduling theory are Makespan Minimization on Unrelated Machines and its dual, the Santa Claus problem (also called Max-Min Fair Allocation). We refer to the two problems as Makespan and Santa Claus, in short. In Makespan, a set of jobs J are available to schedule on a set of machines M, where the processing time of job $j \in J$ on machine $i \in M$ is $p_{i,j}$. We let $p_{i,j} = \infty$ if job j cannot be processed on machine i. The jobs must be assigned non-preemptively, and the goal is to minimize the makespan, i.e., the time the last machine finishes processing. In Santa Claus, a set of items I are available to assign to a set of agents A, where the value agent $i \in A$ receives from item $j \in I$ is $p_{i,j} \geq 0$. Each item can be allocated to at most one agent, and the total value agent i receives is the sum of the values from individual items. The goal is to find an allocation of items to agents so that the minimum total value of any agent is maximized.

For Makespan, the seminal work of Lenstra, Shmoys, and Tardos gave a factor 2 approximation algorithm and showed that the problem is NP-hard to approximate within a factor

*Email: samidavies@berkeley.edu. Department of EECS, UC Berkeley, and RelationalAI.

†Email: venkatg@berkeley.edu. Simons Institute for the Theory of Computing, and Departments of EECS and Mathematics, UC Berkeley. Research supported in part by NSF grants CCF-2228287 and CCF-2211972 and a Simons Investigator award.

‡Email: xuandi_ren@berkeley.edu. Department of EECS, UC Berkeley. Research supported in part by NSF CCF-2228287.

better than 3/2 [LST90] (essentially it is NP-hard to tell if the makespan is 3 or 2 for an instance with nonnegative, integral processing times). Both the hardness and approximation results are still the best known for the general settings. It is NP-hard to approximate Santa Claus within a factor better than 2 [BD05] (similarly to Makespan, it is NP-hard to tell if the value is 2 or 1 for an instance with nonnegative, integral values). The best approximation algorithm for Santa Claus has approximation factor $\tilde{O}(m^{\varepsilon})$ and running time $m^{O(1/\varepsilon)}$, for $\varepsilon = \Omega(\log\log m/\log m)$ [CCK09], where m is the number of items.

For both problems in their most general versions, progress on the hardness and algorithmic fronts has been stuck. On the hardness side, e.g., for Santa Claus, all natural approaches via local gadgets break down when one can allocate half the intended value to agents. Specifically, [BCG09] showed the local gadgets used in all previous reductions can be modified to have *degree two*, i.e., each item is shared by at most two agents. However, [CCK09] gave a $(2+\varepsilon)$-approximation algorithm for this special case, ruling out the hope to get better hardness factor using only such gadgets. On the algorithmic front, the lack of progress is largely due to the absence of good convex programs for either problem. For instance, the *configuration linear program* has an integrality gap of 2 for Makespan [VW14] and $\Omega(\sqrt{n})$ for Santa Claus, for n the number of agents[1].

Given the difficulty in improving algorithms or hardness in the general setting, much work over the past several decades has focused on breaking these barriers in special cases. The notable cases for Santa Claus include the restricted setting [Fei08, AKS17, CM18, CM19, DRZ20, HS23], unrelated graph balancing [BCG09, CCK09, VW14], and Max-Min Degree Arborescence [BCG09, BR23, Bam25]. For Makespan, some of the important cases that have been studied include the restricted setting [Sve11, HO15, JR17], graph balancing [EKS14, VW14, JR19], and in the fixed-parameter tractable regime [MW15]. These special settings are valuable because they help the community identify which aspects of the problems really contribute to their difficulty. For example, one such insight was suggested by Verschae and Wiese [VW14] for Makespan: the configuration linear program has integrality gap $1.833 < 2$ in the restricted setting, but has gap 2 for unrelated graph balancing, thus hinting that perhaps the difficulty in the problem is present in unrelated graph balancing but not in the restricted setting[2]. On the other hand, the unrelated graph balancing setting is much easier to handle for Santa Claus, as there exist several constant factor approximations [BCG09, CCK09], indicating that perhaps the difficulties for both problems are not even the same.

In this paper, we study a different type of relaxation for these problems. For Makespan we allow a $1-\alpha$ fraction of jobs to be rejected (i.e., not scheduled on any machine), for Santa Claus we allow a $1-\alpha$ fraction of agents to be rejected (i.e., receive 0 value). Then we study how this affects the approximation factor β for makespan/value guarantee. While a standard approximation algorithm fixes $\alpha = 1$ and optimizes β, we study the trade-off between the two parameters.

This relaxation of Makespan is the unweighted case of the Maximum General Assignment Problem [FGMS06], for which Feige and Vondrák [FV06] gave an algorithm that achieved $\alpha = 1-1/e+10^{-180}$ and $\beta = 1$. On the other hand, Nutov, Beniaminy and Yuster [NBY06] showed the NP-hardness of $\alpha = 1-\varepsilon$ and $\beta = 1$ for some $\varepsilon > 0$. We remark that [FV06]'s improvement over $1-1/e$ relies on a delicate hierarchy of parameters that has to be carefully balanced, which makes it difficult to turn the 10^{-180} into a larger constant. In this work, we give an algorithm that reaches $\alpha > 1-1/e+0.02$ and $\beta = 3/2$.

Theorem 1. *For the* Makespan *problem, there is a polynomial-time randomized algorithm, which given as input an instance where the optimal makespan for scheduling all n jobs on the m machines is T, schedules in expectation $\frac{6e-5}{6e+1}\cdot n > 0.6533\cdot n > (1-1/e+0.02)\cdot n$ many jobs*

[1] Additionally, there is evidence (but not proof) that even systematic strengthenings of LPs—specifically the Sherali-Adams hierarchy—for Santa Claus may not be strong enough to obtain constant factor approximations in polynomial time [Bam25]

[2] In unrelated graph balancing, every job j has at most two machines where $p_{i,j} < \infty$ but these two processing times may be different, while in the restricted setting, $p_{i,j} \in \{p_j, \infty\}$.

within makespan $\frac{3}{2} \cdot T$.

We highlight that the optimal value T we compare against is the optimal makespan for scheduling *all* jobs, not the (potentially smaller) optimal makespan of scheduling a subset. We also remark that scheduling all jobs in makespan less than $3T/2$ is NP-hard [LST90], so when initiating the study of the trade-off between α and $1 < \beta < 2$, it is natural to consider $\beta = 3/2$.

Allowing some jobs to be rejected is not only a natural variant, but we believe it still captures much of the difficulty in the original problems. We conjecture there are some "transition points" on the curve between α and β, where when one wishes to schedule a slightly larger fraction of jobs (say from α to $\alpha + \delta$, for a small constant $\delta > 0$), a large jump in the approximation factor for the makespan, β, is needed. One regime where it seems difficult to smooth the curve is when $\beta = 1$; it is not clear whether increasing the approximation factor on the makespan slightly allows us to schedule a larger fraction of jobs. Similarly, points near $(\alpha, \beta) = (1, 2)$ are interesting because it seems challenging to design an algorithm that rejects a small constant ε (say $\varepsilon < 1/10$) fraction of jobs and schedules the rest in makespan $(2 - \varepsilon) \cdot T$.

Santa Claus with constrained rejections has been studied algorithmically by Golovin [Gol05], who gave an algorithm which for any $k \in \mathbb{Z}^+$, finds an allocation for Santa Claus where at least a $(1 - 1/k)$ fraction of agents obtain value at least $1/k \cdot T$. In this work, we show the hardness of this problem, although in the high-value regime.

Theorem 2. *For the* Santa Claus *problem, there are universal constants $\delta, \varepsilon > 0$, such that given the existence of an assignment where all n agents receive total value at least T, it's NP-hard to find an assignment where $(1 - \delta) \cdot n$ agents receive total value $(1 - \varepsilon) \cdot T$.*

To prove Theorem 2, we introduce bicriteria Set Packing as an intermediate problem (see Definition 3). In the traditional Set Packing problem, we are given as input a collection $\mathcal{S}$ of sets over a universe U of elements, and the goal is to find the maximum number of disjoint sets from $\mathcal{S}$. In bicriteria Set Packing, instead of being forced to choose a whole set from $\mathcal{S}$ as in the original formulation, we relax the problem to finding large disjoint subsets of the sets in $\mathcal{S}$.

While we motivated bicriteria Makespan and Santa Claus from the long-standing gaps between hardness and approximations, bicriteria Set Packing is itself an interesting combinatorial optimization problem due to the tight hardness results for Set Packing (see Section 1.1). In other words, we know Set Packing is a hard problem, but our understanding of how rigid the hardness is to relaxing some of the constraints is limited. In this vein, we obtain the following algorithmic and hardness results.

Theorem 3. *For the* Set Packing *problem, for any $\delta > 0$, there exists $\varepsilon > 0$ and a polynomial-time randomized algorithm, which given as input a collection $\mathcal{S}$ of n sets over a universe U and the existence of m sets in $\mathcal{S}$ that form a partition of U, finds $m' = (1 - \delta) \cdot m$ many sets $S_1, \ldots, S_{m'}$ together with their subsets $A_1 \subseteq S_1, \ldots, A_{m'} \subseteq S_{m'}$, such that for every $i \in [m']$, $|A_i| \geq \varepsilon \cdot |S_i|$, and the sets $A_1, \ldots, A_{m'}$ are disjoint.*

Theorem 4. *For the* Set Packing *problem, there are universal constants $\varepsilon > 0, \alpha > \beta > 0$, such that given as input a collection $\mathcal{S}$ of n sets over a universe U and the existence of $m = \alpha \cdot n$ sets in $\mathcal{S}$ that form a partition of U, it is NP-hard to find $m' = \beta \cdot n$ many sets $S_1, \ldots, S_{m'}$ in $\mathcal{S}$ together with their subsets $A_1 \subseteq S_1, \ldots, A_{m'} \subseteq S_{m'}$, such that for every $i \in [m'], |A_i| \geq (1 - \varepsilon) \cdot |S_i|$, and the sets $A_1, \ldots, A_{m'}$ are disjoint.*

We note that in bicriteria Set Packing, we assume the existence of m sets partitioning U, while in general Set Packing, this might not be the case.

We give the algorithm for bicriteria Makespan (Theorem 1) in Section 2. We show the hardness for bicriteria Santa Claus (Theorem 2) in Section 3. The algorithm and the hardness for bicriteria Set Packing (Theorem 3 and Theorem 4) are in Section 4 and Section 5.

1.1 Related works

Some of the broader categories for bicriteria objectives in scheduling include (1) minimizing a linear combination of different objectives [ABK01, HSW03, MW15, GWL+23], (2) min-

Table 1: A Table Summarizing Bicriteria Results

	Algorithm	Reference	Hardness	Reference
(α, β)-Makespan (Definition 1)	$(1 - 1/e, 1)$	[FGMS06]	$(1 - \varepsilon, 1)$	[NBY06]
	$(1 - 1/e + 10^{-180}, 1)$	[FV06]		
	$(1 - 1/e + 0.02, 1.5)$	[This work]		
(α, β)-Santa Claus (Definition 2)	$(1 - 1/k, 1/k)$	[Gol05]	$(1 - \delta, 1 - \varepsilon)$	[This work]
(α, β)-Set Packing (Definition 3)	$(1 - \delta, \varepsilon)$	[This work]	$(1 - \delta, 1 - \varepsilon)$	[This work]

imizing one objective while constraining the others [Gol05, SG12, SGK13], and (3) obtaining a Pareto optimal with respect to different objectives [PY00, ABK01, SG12, LG23]. Note that Pareto optimal solutions (category (3)) also satisfy the criteria of categories (1) and (2). Our problems of Makespan and Santa Claus with a constrained number of rejected jobs or agents are most similar to those into category (2). Scheduling with rejections are problems in bicriteria scheduling where one of the objectives is a function of the set of rejected jobs [BLMS+00, HSW03, ABK01, TB06, SGK13, PWW20, LL20, LL21]. In the works studying the trade-off between makespan and the rejection penalty using the Pareto optimality criteria, results either only hold for 2 machines [SG12], or are only polynomial time for a constant number of machines [ABK01, SGK13].

For the classic problem of minimizing the makespan on unrelated machines, it is a long-standing open problem in scheduling theory to develop a polynomial time algorithm that beats the 2-approximation [LST90]. Several lines of work explore additional assumptions on the input that make it easier to break the boundary of 2, including in the FPT setting [HS76, JP99, MW15] and in graph balancing [EKS14, HO15, JR19]. Algorithms in the restricted assignment setting ($p_{i,j} \in \{\infty, p_j\}$) estimate the optimal makespan within factors strictly less than 2 but do not necessarily terminate in polynomial time [Sve11, JR17].

Santa Claus was first introduced in the job-machine scheduling setting [Woe97, ES99]. The configuration LP was introduced for Santa Claus by Bansal and Sviridenko [BS06]. Asadpour and Saberi prove that the CLP has integrality gap $\Omega(\sqrt{n})$ and round the solution to the CLP to obtain a $\tilde{O}(\sqrt{n})$ approximation algorithm [AS07], before the improvement to a $\tilde{O}(m^{\varepsilon})$-approximation algorithm with running time $m^{O(1/\varepsilon)}$, for $\varepsilon = \Omega(\log\log m / \log m)$, by Chakrabarty, Chuzhoy, and Khanna [CCK09]. Recently, Bamas et al. showed that for any $\alpha \geq 2, \varepsilon > 0$, a polynomial-time $(2 - 1/\alpha)$-approximation for Makespan leads to a polynomial-time $(\alpha + \varepsilon)$ approximation for Santa Claus, and under the two-value restriction, they are actually equivalent [BLM+24].

For the bicriteria Makespan problem, Chekuri and Khanna [CK05] observed that the algorithm of Lenstra, Shmoys, and Tardos already schedules at least a $1/2$ of the jobs in makespan T. In the case when $\beta = 1$, bicriteria Makespan is a special case of the Maximum General Assignment Problem. Previous work on the Maximum General Assignment Problem by Fleischer, Goemans, Mirrokni, and Sviridenko [FGMS06] proves one can schedule a $1 - 1/e$ fraction of jobs in makespan T. Feige and Vondrák [FV06] improved the fraction to $1 - 1/e + 10^{-180}$. The best NP-inapproximability ratio of the Maximum General Assignment Problem is 10/11 by Chakrabarty and Goel [CG10]. However, in their construction, the jobs are weighted, which makes it incapable to fit into our setting. It was proven by Nutov, Beniaminy, and Yuster [NBY06] that the unit weight case (a.k.a. maximizing the number of jobs scheduled) is also APX-hard.

For bicriteria Santa Claus, Golovin proposed an algorithm which for any $k \in \mathbb{Z}^+$, finds an allocation for Santa Claus where a $(1 - 1/k)$ fraction of agents obtain value $1/k \cdot T$ [Gol05].

For the Set Packing problem where we are given as input a collection $\mathcal{S}$ of sets over a universe U, the goal is to find the maximum number of disjoint sets from $\mathcal{S}$. By the simple correspondence between Independent Set and Set Packing, there is a polynomial time $O\left(\frac{|\mathcal{S}|(\log\log|S|)^2}{(\log|S|)^3}\right)$-approximation algorithm [Fei04] for it, and it cannot be approximated within a factor less than $\Omega(|\mathcal{S}|^{1-\varepsilon})$ for any $\varepsilon > 0$ unless P=NP [Hås99, Zuc06]. In terms of the universe size $|U|$, Halldórsson, Kratochvíl and Telle gave an algorithm that admits an $O(\sqrt{|U|})$ approximation [HKT00], which matches the Independent Set-based hardness $\Omega(|U|^{1/2-\varepsilon})$ [Hås99, Zuc06]. The special case where each set has a bounded size k (known as k-Set Packing) is also widely studied [HS89, Hal95, Ber00, LOSZ20, Neu21, Neu23]. The state-of-the-art approximation ratios are $O(k)$ [Cyg13, LSV13], while the hardness was recently improved from the long-standing $\Omega(k/\log k)$ [HSS06] to $\Omega(k)$ [LST24].

1.2 Preliminaries

We begin with preliminaries for Makespan. It is helpful to consider the bipartite graph $G = (V, E)$ where $V = M \dot\cup J$ and E is the set of (machine, job) pairs (i, j) where $p_{i,j} < \infty$. A *schedule* $\mathcal{O}$ is simply a subset of E. T denotes the optimal makespan. In our algorithms for (α, β)-bicriteria Makespan, we partition E into disjoint sets of *large* and *small* edges, E_L and E_S, according to the processing times $E_L = \{(i, j) \mid i \in M, j \in J, \frac{T}{2} < p_{i,j} \leq T\}$, and $E_S = \{(i, j) \mid i \in M, j \in J, 0 \leq p_{i,j} \leq \frac{T}{2}\}$, and define G_L and G_S to be the bipartite graphs induced by E_L and E_S, respectively. We let $m^\star$ denote the size of a maximum matching in G_L. When we input an instance of Makespan into an algorithm for (α, β)-bicriteria Makespan, we denote the input with the shorthand $\Gamma = (J, M, T, E')$, for J the jobs, M the machines, T the optimal makespan, and $E' \subseteq E$ the edges along which we make assignments.

Definition 1 (Makespan Problem)**.** *In the* Makespan *problem, we are given as input a set of m machines M, a set of n jobs J, and a table $\{p_{i,j}\}_{i\in M, j\in J}$, where $p_{i,j}$ is the processing time of job j on machine i. We write $p_{i,j} = \infty$ if job j cannot be scheduled on machine i. The goal is to assign jobs to machines, such that the maximum total processing time on any machine is minimized.*

*For $0 \leq \alpha \leq 1 \leq \beta \leq 2$, (α, β)-*bicriteria Makespan *is the task where given the optimal makespan T, the goal is to schedule a subset of $\alpha \cdot n$ jobs on M within makespan $\beta \cdot T$.*

The definitions for Santa Claus and Set Packing and their bicriteria variants are as follows. Other notations are self-contained in those relevant sections.

Definition 2 (Santa Claus Problem)**.** *In the* Santa Claus *problem, we are given as input a set of n agents A, a set of m items I, and a table $\{p_{i,j}\}_{i\in A, j\in J}$ where $p_{i,j}$ is the value agent i can get on receiving item j. The goal is to assign items to agents to maximize the minimum total value of any agent.*

*For $0 \leq \alpha, \beta \leq 1$, (α, β)-*bicriteria Santa Claus *is the task where given the optimum value T, the goal is to allocate items to agents so that at least $\alpha \cdot n$ agents have total value at least $\beta \cdot T$.*

Definition 3 (Set Packing Problem)**.** *In the* Set Packing *problem, we are given as input a collection of sets $\mathcal{S}$ over a universe U, and the goal is to find the maximum number of disjoint sets in $\mathcal{S}$.*

*For $0 \leq \alpha, \beta \leq 1$, (α, β)-*bicriteria Set Packing *is the task where given the guarantee that there are m sets in $\mathcal{S}$ partitioning U, the goal is to find $m' = \alpha \cdot m$ many sets $S_1, \ldots, S_{m'}$ in $\mathcal{S}$ together with their subsets $A_1 \subseteq S_1, \ldots, A_{m'} \subseteq S_{m'}$, such that for every $i \in [m']$, $|A_i| \geq \beta \cdot |S_i|$, and the sets $A_1, \ldots, A_{m'}$ are disjoint.*

2 An Algorithm for $(0.6533, 1.5)$-bicriteria Makespan

In this section, we prove Theorem 1.

Theorem 1. *For the* Makespan *problem, there is a polynomial-time randomized algorithm, which given as input an instance where the optimal makespan for scheduling all n jobs on the m machines is T, schedules in expectation $\frac{6e-5}{6e+1} \cdot n > 0.6533 \cdot n > (1 - 1/e + 0.02) \cdot n$ many jobs within makespan $\frac{3}{2} \cdot T$.*

Our algorithm has three sub-procedures:

1. a deterministic Algorithm 1, which simply computes a maximum matching on G_L and schedules these $m^\star$ jobs within makespan T;
2. a randomized Algorithm 2, which given the promise that all jobs can fit into makespan T, schedules a $(1 - \frac{1}{e})$ fraction of jobs in expectation within makespan T;
3. a deterministic Algorithm 3, which schedules $\frac{1}{6}(n - m^\star)$ many jobs within makespan $\frac{T}{2}$, using only edges in E_S.

While each sub-procedure on its own is simple (especially Algorithm 1 and Algorithm 2), the salient point is that they can be combined in a way that is stronger than any of them individually.

We describe each algorithm, then show how to combine them in Section 2.4.

2.1 Algorithm 1: The Matching Algorithm on Large Jobs

Algorithm 1 simply finds a matching in G_L and schedules these $m^\star$ jobs. Given the result of Algorithm 1, the following Lemma 1 shows a lower bound on the number of jobs that can be scheduled using only small edges.

Algorithm 1 Matching Large Edges

1: **Input**: A Makespan instance $\Gamma = (J, M, T, E_L)$.
2: Find a maximum matching $\mathcal{O}^\star$ on the bipartite graph $G_L = (V = M \dot\cup J, E = E_L)$.
3: Output $\mathcal{O}^\star$.

Lemma 1. *It is possible to schedule at least $n - m^\star$ many jobs within makespan T using only edges in E_S, where $m^\star$ is the matching size output by Algorithm 1.*

Proof. Recall the guarantee that it's possible to schedule all n jobs of J on M within makespan T. Let $\mathcal{O} \subseteq E$ be such a schedule. For every machine $i \in M$, there is at most one job j scheduled on it in $\mathcal{O}$ with $(i, j) \in E_L$, as otherwise the makespan will exceed T. Therefore, $\mathcal{O} \cap E_L$ forms a matching on G_L. Since the matching output by Algorithm 1 is a maximum matching, we have $m^\star \geq |\mathcal{O} \cap E_L|$.

By simply throwing out edges in $\mathcal{O} \cap E_L$ from the schedule $\mathcal{O}$, we prove the existence of a schedule using only E_S, as

$$|\mathcal{O} \cap E_S| = |\mathcal{O}| - |\mathcal{O} \cap E_L| \geq |\mathcal{O}| - m^\star = n - m^\star. \qquad \square$$

2.2 Algorithm 2: A Randomized LP-rounding Algorithm

We define the *Configuration LP* for Makespan, which we sometimes abbreviate as the CLP. For machine $i \in M$, let $\mathcal{C}_i$ be the subsets of jobs that have combined processing time at most T on machine i, i.e. $\mathcal{C}_i = \{C \subseteq J \mid \sum_{j \in C} p_{i,j} \leq T\}$. Let $y_{i,C}$ be the variable indicating whether configuration C is scheduled on machine i. Then the configuration LP is the following:

$$\sum_{C \in \mathcal{C}_i} y_{i,C} = 1 \; \forall i \in M, \quad \sum_{i \in M} \sum_{C \in \mathcal{C}_i : j \in C} y_{i,C} = 1 \; \forall j \in J, \quad y_{i,C} \geq 0 \; \forall i \in M, \forall C \in \mathcal{C}_i. \tag{1}$$

If all jobs can be scheduled within makespan T in an integral solution, then there is a feasible solution to the CLP. Further, a feasible solution to the CLP can be found to any degree of accuracy in polynomial time [BS06, Sve11].

Given a solution to the CLP, the rounding algorithm simply lets each machine sample a configuration C in $\mathcal{C}_i$ with probability $y_{i,C}$, then schedule all unscheduled jobs that it samples. See the pseudo-code in Algorithm 2.

Algorithm 2 Randomized Rounding of the CLP

1: **Input**: A MAKESPAN instance $\Gamma = (J, M, T, E)$.
2: Solve the CLP defined in Equation (1).
3: Initialize $\mathcal{O}^\star = \emptyset$.
4: **for** every machine $i \in M$ **do**
5: Sample a configuration $C \in \mathcal{C}_i$ with probability $y_{i,C}$.
6: **for** every job j in the sampled configuration **do**
7: Add (j, i) to $\mathcal{O}^\star$ if j is not yet scheduled in $\mathcal{O}^\star$.
8: Output $\mathcal{O}^\star$.

Lemma 2. *Given the promise that all jobs can be integrally scheduled within makespan T, Algorithm 2 schedules at least a $(1 - \frac{1}{e})$ fraction of the jobs in expectation within makespan T.*

Proof. A job is not scheduled only if it is not sampled by any machine. Thus,

$$\begin{aligned}
\mathbb{E}[\text{number of scheduled jobs}] &= \sum_{j \in J} \mathbb{P}[j \text{ sampled}] \\
&= \sum_{j \in J} \left(1 - \prod_{i \in M} \mathbb{P}[j \text{ not sampled on machine } i]\right) \\
&= \sum_{j \in J} \left(1 - \prod_{i \in M} \left(1 - \sum_{C \in \mathcal{C}_i : j \in C} y_{i,C}\right)\right) \\
&\geq \sum_{j \in J} \left(1 - e^{-\sum_{i \in M} \sum_{C \in \mathcal{C}_i : j \in C} y_{i,C}}\right) \\
&= \sum_{j \in J} \left(1 - \frac{1}{e}\right) = \left(1 - \frac{1}{e}\right) \cdot n.
\end{aligned}$$

□

We note that this result was known from [FGMS06].

2.3 Algorithm 3: A Greedy Algorithm using only Small Jobs

Our Algorithm 3 deterministically schedules $\frac{1}{6}(n - m^\star)$ many jobs within makespan $\frac{T}{2}$ using only edges in E_S, where $m^\star$ is the maximum matching size on G_L as computed by Algorithm 1. Algorithm 3 first computes a schedule of makespan T by greedily picking the job with minimum processing time[3], then truncates each machine's schedule up to time $\frac{T}{2}$.

Lemma 3. *Let $m^\star$ be maximum matching size on G_L computed by Algorithm 1. Algorithm 3 schedules at least $\frac{1}{6}(n - m^\star)$ many jobs within makespan $\frac{T}{2}$.*

We split the proof of Lemma 3 into the following Lemma 4 and Lemma 5:

Lemma 4. *At the beginning of Line 13 in Algorithm 3, we have $\sum_{i \in M} |\ell_i| \geq \frac{1}{2}(n - m^\star)$.*

To prove Lemma 4, we show in a recursive way that in the `while` loop in Line 5-12, starting with any available job set R and any vector $c = (c_1, \ldots, c_m)$ of current completion times of all machines, our algorithm adds at least half of the jobs to the lists as compared to any other feasible schedule. This is formalized in the following Proposition 1.

[3]This is just a specific instantiation of the classic *List Scheduling* algorithm [Gra69].

Algorithm 3 Deterministic Greedy Algorithm

Input: A MAKESPAN instance $\Gamma = (J, M, T, E_S)$.
Initialize $c_i \leftarrow 0, d_i \leftarrow 0$ for all $i \in M$.
Set up an empty list ℓ_i for all $i \in M$.
Initialize R to be J, the set of all jobs.
while $R \neq \emptyset$ **do**
    Let $p_{i^\star,j^\star}$ be the minimum $p_{i,j}$ such that $j \in R$ and $c_i + p_{i,j} \leq T$.
    if Such a $(i^\star, j^\star)$ exists **then**
        Append $j^\star$ to the list $\ell_{i^\star}$.
        $c_{i^\star} \leftarrow c_{i^\star} + p_{i^\star,j^\star}$.
        Remove $j^\star$ from R.
    else
        break
Initialize $\mathcal{O}^\star = \emptyset$.
for every machine $i \in M$ **do**
    Sort the list ℓ_i according to increasing $p_{i,j}$.
    for job j in the list ℓ_i **do**
        if $d_i + p_{i,j} \leq \frac{T}{2}$ **then**
            Add (j, i) to $\mathcal{O}^\star$.
            Update $d_i \leftarrow d_i + p_{i,j}$.
        else
            break
Output $\mathcal{O}^\star$.

Proposition 1. *Let $R \subseteq J$ be an arbitrary set of jobs and $c = (c_1, \ldots, c_m)$, where $\forall i \in M, 0 \leq c_i \leq T$, be a vector of the current completion times of the machines. If it's possible to schedule a set $S \subseteq R$ of jobs continuing from c within makespan T, then the while loop in Algorithm 3, starting with configuration (R, c), adds at least $\frac{1}{2}|S|$ many jobs to the lists.*

Proof. If the maximum scheduleable set $S = \emptyset$, there is nothing to show. Otherwise, fix a choice of the maximum S and an arbitrary schedule $\mathcal{O}$ of S on the machines continuing from c. Let $(i^\star, j^\star)$ be the (machine, job) pair picked by Algorithm 3, which has minimum processing time among all available pairs.

Suppose $j^\star \notin S$, i.e., $\mathcal{O}$ does not schedule $j^\star$ anywhere. Then $\mathcal{O}$ must have scheduled some other job $j' \in S$ on machine $i^\star$, since, otherwise, $\mathcal{O} \cup \{(i^\star, j^\star)\}$ is a valid schedule, contradicting the maximality of S. Furthermore, $p_{i^\star,j^\star} \leq p_{i^\star,j'}$ as scheduling j' on $i^\star$ was a valid option but we chose the smallest available processing time. Thus, $S \setminus \{j'\} \cup \{j^\star\}$ is also a maximum scheduleable set with $O \setminus \{(i^\star, j')\} \cup \{(i^\star, j^\star)\}$ being the schedule.

In the following, we assume without loss of generality that $j^\star \in S$. Again, we can without loss of generality assume $\mathcal{O}$ schedules some job j' on machine $i^\star$ (j' could be another job or $j^\star$ itself). Given $p_{i^\star,j^\star} \leq p_{i^\star,j'}$, it's possible to schedule the set $S \setminus \{j^\star, j'\}$, which is a subset of $R' = R \setminus \{j^\star\}$, starting with the completion times $c' = (c_1, \ldots, c_{i^\star} + p_{i^\star,j^\star}, \ldots, c_m)$. Invoking Proposition 1 on (R', c'), we have at least $\frac{1}{2}|S \setminus \{j^\star, j'\}|$ many jobs in the lists. Adding the job $j^\star$ gives us $\frac{1}{2}|S \setminus \{j^\star, j'\}| + 1 \geq \frac{1}{2}|S|$, which finishes the proof. □

Given Proposition 1, Lemma 4 follows by plugging the set of $n - m^\star$ jobs guaranteed by Lemma 1 into the set S in Proposition 1, and setting $c = (0, \ldots, 0)$.

Lemma 5. *Line 13 $\sim$ Line 20 of Algorithm 3 schedules $\frac{1}{3}\sum_{i \in M} |\ell_i|$ jobs within makespan $\frac{T}{2}$.*

Proof. Consider each machine i. If $|\ell_i| = 1$, since all edges are in E_S, every $p_{i,j}$ is at most $\frac{T}{2}$ and we can always schedule that unique job on machine i.

If $|\ell_i| \geq 2$, then given $\sum_{j \in \ell_i} p_{i,j} \leq T$, the smaller $\lfloor \frac{|\ell_i|}{2} \rfloor$ many jobs have total processing time at most $\frac{T}{2}$. If $|\ell_i| = 2$, then $\lfloor \frac{|\ell_i|}{2} \rfloor = \frac{|\ell_i|}{2}$. Otherwise for $|\ell_i| \geq 3$, we have $\lfloor \frac{|\ell_i|}{2} \rfloor \geq \frac{|\ell_i|-1}{2} \geq \frac{|\ell_i|}{2} - \frac{|\ell_i|}{6} = \frac{|\ell_i|}{3}$. □

Lemma 3 follows from combining Lemma 4 and Lemma 5.

We note that Lemma 3 is quite brittle with the makespan of $T/2$. One may hope it is possible to schedule a smaller fraction of jobs in a smaller makespan and have a smooth trade-off here, which would necessitate the processing time used to define the edges in E_S be smaller. However if we partition the edges in E into more subsets, or if we make the threshold processing time between E_L and E_S smaller, we lose the guarantee that at most one edge in $E \setminus E_S$ can be integrally scheduled per machine. Overall, it's not clear how to make this trade-off worth it in the analysis so that a higher fraction of jobs can be scheduled.

2.4 The Combined Algorithm

In this subsection, we show how to combine the 3 algorithms to prove Theorem 1. The final algorithm outputs the better one of the following two schedules:

(S1) The schedule output by Algorithm 1.

(S2) The schedule obtained by first running Algorithm 3, then Algorithm 2 on the remaining jobs.

Proof of Theorem 1. Item **(S1)** schedules $m^\star$ jobs within makespan T.

We now analyze Item **(S2)**. By Lemma 3, Algorithm 3 schedules at least $\frac{1}{6}(n - m^\star)$ many jobs within makespan $\frac{T}{2}$. Recall the guarantee that there is a way to schedule all n jobs within makespan T, thus it's also possible for the remaining $n - \frac{1}{6}(n - m^\star)$ many jobs to fit into makespan T. By Lemma 2, Algorithm 2 schedules at least a $(1 - \frac{1}{e})$ fraction of them in expectation.

Combining Item **(S1)** and Item **(S2)**, we schedule[4]

$$\max\left(m^\star, \frac{1}{6}(n - m^\star) + \left(1 - \frac{1}{e}\right)\left(n - \frac{1}{6}(n - m^\star)\right)\right)$$

many jobs within makespan $\frac{3}{2}T$. The minimum is attained when $m^\star = \frac{6e-5}{6e+1} \cdot n$. □

3 Hardness for $(1 - \delta, 1 - \varepsilon)$-bicriteria Santa Claus

In this section, we prove Theorem 2.

Theorem 2. *For the* Santa Claus *problem, there are universal constants $\delta, \varepsilon > 0$, such that given the existence of an assignment where all n agents receive total value at least T, it's NP-hard to find an assignment where $(1 - \delta) \cdot n$ agents receive total value $(1 - \varepsilon) \cdot T$.*

We reduce from the following hardness of $(\frac{\beta}{\alpha}, 1 - \varepsilon)$-bicriteria Set Packing.

Theorem 4. *For the* Set Packing *problem, there are universal constants $\varepsilon > 0, \alpha > \beta > 0$, such that given as input a collection $\mathcal{S}$ of n sets over a universe U and the existence of $m = \alpha \cdot n$ sets in $\mathcal{S}$ that form a partition of U, it is NP-hard to find $m' = \beta \cdot n$ many sets $S_1, \ldots, S_{m'}$ in $\mathcal{S}$ together with their subsets $A_1 \subseteq S_1, \ldots, A_{m'} \subseteq S_{m'}$, such that for every $i \in [m'], |A_i| \geq (1 - \varepsilon) \cdot |S_i|$, and the sets $A_1, \ldots, A_{m'}$ are disjoint.*

We first prove Theorem 2 given Theorem 4. The proof of Theorem 4 is in Section 5.

[4] A natural question is whether one could use the 2-approximation of Lenstra, Shmoys, and Tardos (or a subroutine of it) to schedule a large set of jobs in makespan $T/2$, then combine this with Algorithm 2. We found that such a procedure might do better for some $m^\star$ values, but it does not help the worst-case.

Proof of Theorem 2. We set $\delta = \alpha - \beta$, and reduce a Set Packing instance as in Theorem 4 to a Santa Claus instance as follows.

- Set up n agents which correspond to the n sets.
- Set up $|U| + (1-\alpha)\cdot n$ items, where $|U|$ of them are normal items corresponding to the elements in the universe, and the remaining $(1-\alpha)\cdot n$ items are dummy items.
- For an agent i that corresponds to a set S_i and a normal item j that corresponds to an element $j \in U$, we have $p_{i,j} = T/|S_i|$ if $j \in S_i$ and 0 otherwise. For any agent i and any dummy item j, we set $p_{i,j} = T$.

For completeness, we can let the $m = \alpha \cdot n$ agents that correspond to the m non-intersecting sets take their interested items, and distribute a dummy item to each of the remaining $(1-\alpha)\cdot n$ agents.

For soundness, suppose there are $(1-\delta)\cdot n = (1-\alpha+\beta)\cdot n$ many agents who get happiness at least $(1-\varepsilon)\cdot T$. Among them at most $(1-\alpha)\cdot n$ agents are satisfied by the dummy items, so at least $m' = \beta \cdot n$ agents get happiness $(1-\varepsilon)\cdot T$ by only picking normal items. This gives us a solution to the $(\frac{\beta}{\alpha}, 1-\varepsilon)$-bicriteria Set Packing. □

4 An Algorithm for $(1-\delta, \varepsilon)$-bicriteria Set Packing

In this section, we prove Theorem 3.

Theorem 3. *For the* Set Packing *problem, for any $\delta > 0$, there exists $\varepsilon > 0$ and a polynomial-time randomized algorithm, which given as input a collection $\mathcal{S}$ of n sets over a universe U and the existence of m sets in $\mathcal{S}$ that form a partition of U, finds $m' = (1-\delta)\cdot m$ many sets $S_1, \ldots, S_{m'}$ together with their subsets $A_1 \subseteq S_1, \ldots, A_{m'} \subseteq S_{m'}$, such that for every $i \in [m']$, $|A_i| \geq \varepsilon \cdot |S_i|$, and the sets $A_1, \ldots, A_{m'}$ are disjoint.*

We first introduce the following version of a Chernoff Bound.

Lemma 6 (Multiplicative Chernoff Bound)**.** *Suppose $x_1, \ldots, x_n$ are independent random variables taking values in $\{0,1\}$. Let X denote their sum and let $\mu = \mathbb{E}[X]$ denote the sum's expected value. Then for any $\gamma > 0$, $\Pr[X \geq (1+\gamma)\mu] \leq \exp\left(-\frac{\gamma^2\mu}{2+\gamma}\right)$, and for any $0 < \gamma < 1$, $\Pr[X \leq (1-\gamma)\mu] \leq \exp\left(-\gamma^2\mu/2\right)$.*

Fix any $\delta > 0$, we set constants $D = 40/\delta$, $C = 400/\delta^2$, and $\varepsilon = \delta^2/800$. We then divide the sets into two groups based on whether their sizes exceed C. Specifically, let $\mathcal{S}_s = \{S \in \mathcal{S} \mid |S| \leq C\}$ and $\mathcal{S}_b = \{S \in \mathcal{S} \mid |S| > C\}$. Note that as $\varepsilon < 1/C$, for a set in $\mathcal{S}_s$, picking 1 element suffices to represent an ε fraction of it. Therefore, we first allocate 1 element to as many sets in $\mathcal{S}_s$ as possible using a Max-flow algorithm (Algorithm 5), then run an LP-rounding based algorithm (Algorithm 6) on the remaining instance. See the following pseudo-codes for the description of the algorithms. Throughout this section, when we consider some subproblem $(\mathcal{S}', U')$, we mean one is only allowed to pick sets in $\mathcal{S}'$ that are *fully contained* in U'.

Algorithm 4 A Polynomial-time Randomized Algorithm for $(1-\delta, \varepsilon)$-Set Packing

Input: a Set Packing instance $\Gamma = (\mathcal{S}, U)$, where $\mathcal{S} = \mathcal{S}_s \dot\cup \mathcal{S}_b$.
Run Algorithm 5 on $(\mathcal{S}_s, U)$ and let $(\mathcal{T}_s, U_s)$ be the output.
Run Algorithm 6 on $(\mathcal{S}_b, U \setminus U_s)$ and let $(\mathcal{T}_b, U_b)$ be the output.
Output: $\mathcal{T}_s \cup \mathcal{T}_b$.

The following lemma shows the optimality of working in two phases as in Algorithm 4. For a Set Packing instance $\Gamma = (\mathcal{S}, U)$, we use $\mathrm{OPT}(\Gamma)$ to denote the maximum number of disjoint sets in it.

Algorithm 5 An Algorithm for the Small-sized Sets

Input: A collection of sets $\mathcal{S}$ over the universe U, where every set has size $\leq C$.
Build the network G containing a source s, a sink t, a node for every $S \in \mathcal{S}$, and a node for every $u \in U$.
Add edge (s, S) with capacity 1 to G for every $S \in \mathcal{S}$.
Add edge (u, t) with capacity 1 to G for every $u \in U$.
Add edge (S, u) with capacity 1 to G for every $u \in S$.
Run a Max-flow algorithm on G.
Let $\mathcal{T}$ be the collection of sets S with edge (s, S) saturated.
Let U' be the collection of elements u with edge (u, t) saturated.
Output: $(\mathcal{T}, U')$.

Lemma 7. *Let $\Gamma = (\mathcal{S}, U)$ be a* Set Packing *instance. For any $U' \subseteq U$, we have $|U'| + OPT(\mathcal{S}, U \setminus U') \geq OPT(\mathcal{S}, U)$.*

Proof. Starting with the optimal solution for $(\mathcal{S}, U)$, removing an element from U only affects at most one set since the picked sets are disjoint. Doing this for every element in U' finishes the proof. □

Let $(\mathcal{T}_s, U_s)$ be the output of Algorithm 5. Clearly $|\mathcal{T}_s| = |U_s|$. By Lemma 7, we have $|\mathcal{T}_s| + \mathrm{OPT}(\mathcal{S}, U \setminus U_s) \geq \mathrm{OPT}(\mathcal{S}, U)$. By the optimality of Algorithm 5, every set in $\mathcal{S}_s$ must intersect U_s, thus $\mathrm{OPT}(\mathcal{S}, U \setminus U_s) = \mathrm{OPT}(\mathcal{S}_b, U \setminus U_s)$. Note that Algorithm 6 will solve the following LP:

$$\text{maximize} \quad \sum_S x_S \qquad \text{s.t.} \quad \sum_{S \ni u} x_S \leq 1 \quad \forall u \in U, \quad x_S \in [0,1] \quad \forall S \in \mathcal{S}. \tag{2}$$

Algorithm 6 An Algorithm for the Large-sized Sets

Input: A collection of sets $\mathcal{S}$ over the universe U, where every set has size $> C$. Parameter D.
Solve LP (2) and let x be an optimal solution.
Initialize $\mathcal{A} = \emptyset$.
Add S to $\mathcal{A}$ with probability x_S for all $S \in \mathcal{S}$.
Let $T = \{u \in U \mid |\{S \in \mathcal{A} \mid S \ni u\}| \geq D\}$, i.e., the elements that are covered $\geq D$ times by $\mathcal{A}$.
Let $\mathcal{B} = \{S \in \mathcal{A} \mid |S \cap T| < 0.1 \cdot S\}$, i.e., the sets in $\mathcal{A}$ that only contain few elements in T.
Allocate each $u \in U \setminus T$ to a set in $\mathcal{B}$ that contains it independently at random.
Let $\mathcal{C} = \{S \in \mathcal{B} \mid S \text{ collects } \geq \varepsilon \cdot |S \cap T| \text{ many elements}\}$.
Let U' be the elements collected by sets in $\mathcal{C}$.
Output: $(\mathcal{C}, U')$.

Proof of Theorem 3. Given the promise that there are m disjoint sets in the Set Packing instance, we have $\sum_S x_S \geq m$ where $\{x_S\}$ is an optimal solution to LP (2). Let y_S be the 0-1 random variable indicating whether or not S is picked into $\mathcal{A}$, where $y_S = 1$ with probability x_S.

Fix any element $u \in U$, we have $\mathbb{E}[\sum_{S \ni u} y_S] = \sum_{S \ni u} x_S \leq 1$, i.e., in expectation it will be covered at most once. Consider the probability that it is covered at least D times, for some large enough constant $D = D(\delta)$. By the Chernoff bound (Lemma 6) ,

$$\begin{aligned}
\Pr\left[\sum_{S \ni u} y_S \geq D\right] &\leq \exp\left(-\frac{(D-1)^2}{2+(D-1)}\right) && \text{(Lemma 6 with } \gamma = D-1 \text{ and } \mu = 1) \\
&\leq \delta/20. && \text{(Since } D = 40/\delta)
\end{aligned}$$

Recall that $T = \{u \mid \sum_{S \ni u} y_S \geq D\}$. Thus for any fixed u, we have $\Pr[u \in T] \leq \delta/20$.

We bound the probability that a set is in $\mathcal{A} \setminus \mathcal{B}$, i.e., is discarded because of containing too many "popular" elements. For any fixed set S, we have $\mathbb{E}[|S \cap T|] \leq \delta/20 \cdot |S|$, by linearity of expectation and $\Pr[|S \cap T| \geq 0.1 \cdot |S|] \leq \delta/2$. by Markov's inequality. Therefore,

$$\begin{aligned}
\mathbb{E}[|\mathcal{B}|] &= \sum_{S \in \mathcal{S}} \Pr[S \in \mathcal{A}] \cdot \Pr[S \in B \mid S \in \mathcal{A}] \\
&= \sum_{S \in \mathcal{S}} \Pr[S \in \mathcal{A}] \cdot \Pr[|S \cap T| < 0.1 \cdot |S| \mid S \in \mathcal{A}] \\
&\geq (1 - \delta/2) \cdot \sum_{S \in \mathcal{S}} \Pr[S \in \mathcal{A}] \\
&= (1 - \delta/2) \cdot \mathbb{E}\left[|\mathcal{A}|\right],
\end{aligned}$$

which means in expectation, we do not lose too much by discarding the popular elements and only focusing on the sampled sets that contain mostly non-popular elements.

From now on we only focus on the non-popular elements $U \setminus T$, i.e., the elements which are covered no more than D times. We argue that by allocating each element $u \in U \setminus T$ to a random set $S \in \mathcal{B}$ that contains it, most sets will collect a reasonable fraction of their elements.

For a fixed set $S \in \mathcal{B}$ and an element $u \in S \setminus T$, since u is covered $\leq D$ times we have

$$\Pr[u \text{ is allocated to } S] \geq 1/D.$$

Summing this over $S \setminus T$, we use linearity of expectation

$$\mathbb{E}[\text{the number of elements } S \text{ collects}] \geq (1/D) \cdot |S \setminus T|.$$

Recall that our input sets have size $> C$. Fix a set $S \in \mathcal{B}$, we have $|S \setminus T| \geq 0.9 \cdot |S| \geq 0.9 \cdot C$. Again using Lemma 6 with $\gamma = 1 - \varepsilon \cdot D$ and $\mu = (1/D) \cdot |S \setminus T| \geq 0.9 \cdot C/D$,

$$\begin{aligned}
\Pr[S \text{ collects} < \varepsilon \cdot |S \setminus T| \text{ elements}] &\leq \exp\left(-\frac{(1 - \varepsilon \cdot D)^2 \cdot 0.9 \cdot C/D}{2}\right) \\
&\leq \exp\left(-\frac{0.5 \cdot 0.9 \cdot C/D}{2}\right) \\
&< \delta/2.
\end{aligned}$$

The second to last inequality follows due to the fact that $(1 - \varepsilon \cdot D)^2 \geq 0.5$, and the last inequality since $C/D \geq 10/\delta$. Therefore,

$$\begin{aligned}
\mathbb{E}[|\mathcal{C}|] &= \sum_{S \in \mathcal{S}} \Pr[S \in \mathcal{B}] \cdot \Pr[S \in \mathcal{C} \mid S \in \mathcal{B}] \\
&= \sum_{S \in \mathcal{S}} \Pr[S \in \mathcal{B}] \cdot \Pr[S \text{ collects} \geq \varepsilon \cdot |S \setminus T| \text{ many elements} \mid S \in \mathcal{B}] \\
&\geq (1 - \delta/2) \cdot \sum_{S \in \mathcal{S}} \Pr[S \in \mathcal{B}] = (1 - \delta/2) \cdot \mathbb{E}[|\mathcal{B}|] \\
&\geq (1 - \delta/2)^2 \cdot \mathbb{E}[|\mathcal{A}|] = (1 - \delta/2)^2 \cdot \sum_{S \in \mathcal{S}} x_S \geq (1 - \delta) \cdot m,
\end{aligned}$$

which means in expectation, at least $(1 - \delta) \cdot m$ many sets S get at least $\varepsilon \cdot |S \setminus T| \geq 0.9 \cdot \varepsilon \cdot |S|$ many elements. □

5 Hardness of $(1 - \delta, 1 - \varepsilon)$-bicriteria Set Packing

In this section, we prove Theorem 4.

Theorem 4. *For the* Set Packing *problem, there are universal constants $\varepsilon > 0, \alpha > \beta > 0$, such that given as input a collection $\mathcal{S}$ of n sets over a universe U and the existence of $m = \alpha \cdot n$ sets in $\mathcal{S}$ that form a partition of U, it is NP-hard to find $m' = \beta \cdot n$ many sets $S_1, \ldots, S_{m'}$ in $\mathcal{S}$ together with their subsets $A_1 \subseteq S_1, \ldots, A_{m'} \subseteq S_{m'}$, such that for every $i \in [m'], |A_i| \geq (1-\varepsilon) \cdot |S_i|$, and the sets $A_1, \ldots, A_{m'}$ are disjoint.*

To ease the notation, we refer to the property of sets $S_1, \ldots, S_{m'}$ in Theorem 4 as *ε-almost disjoint*. Specifically, $S_1, \ldots, S_{m'}$ are ε-almost disjoint if and only if there exist sets $A_1 \subseteq S_1, \ldots, A_{m'} \subseteq S_{m'}$, such that for every $i \in [m'], |A_i| \geq (1-\varepsilon) \cdot |S_i|$, and $A_1, \ldots, A_{m'}$ are disjoint.

We use the canonical reduction from q-CSP to Set Packing [HSS06,LST24], while replacing the gadget in [LST24] with Feige's hypercube partition system [Fei98], and plugging in Feige's hardness of degree-5 3-CNF [Fei98]. We first give the definition of CSP, then show the reduction from CSP to Set Packing and give the analysis.

Definition 4 (CSP)**.** *A q-CSP instance $\Pi = (G = (V, E), \Sigma, C = \{c_e\}_{e \in E})$ consists of $n = |V|$ variables and $m = |E|$ constraints, where each constraint $e \in E$ is on q variables and is defined by the function $c_e : \Sigma^q \to \{0, 1\}$. An assignment is a function from V to Σ. The value of Π is the maximum fraction of constraints that can be satisfied by some assignment.*

Reduction. Given a d-regular q-CSP instance over alphabet Σ, we construct a hypergraph gadget $G^{(x)} = (V^{(x)}, E^{(x)})$ for each variable x with the following properties:

1. $E^{(x)}$ is partitioned into $|\Sigma|$ matchings $E_1^{(x)}, \ldots, E_{|\Sigma|}^{(x)}$ with $|E_1^{(x)}| = \ldots = |E_{|\Sigma|}^{(x)}| = d$, where $E_i^{(x)} = \{e_{i,j}^{(x)}\}_{j \in [d]}$.
2. Any two hyperedges from two different matchings, i.e., any $e_{i_1,j_1}^{(x)}$ and $e_{i_2,j_2}^{(x)}$ for $i_1 \neq i_2$ and $j_1, j_2 \in [d]$, must intersect in a reasonable fraction of their sizes.

With this gadget, the only way to pick almost disjoint sets is to fix some $v \in \Sigma$, and take sets (a.k.a. hyperedges) from the matching $E_v^{(x)}$. This corresponds to the case where variable x is assigned value v in the CSP instance. For $j \in [d]$, the j-th hyperedge in $E_i^{(x)}$ corresponds to the j-th constraint involving x, assuming there is an arbitrary order of all constraints. In the final construction, we consider each constraint φ on variables $(x_1, \ldots, x_q)$ and a satisfying assignment $(v_1, \ldots, v_q) \in \Sigma^q$, and build a set S for them. Assuming φ is the j_i-th constraint for variable x_i, the set S is constructed to be

$$S = \bigcup_{i \in [q]} e_{v_i, j_i}^{(x_i)}.$$

Suppose the CSP instance has m constraints and the average number of satisfying assignments for each constraints is η, then the total number of constructed sets is $\eta \cdot m$.

Completeness. Suppose there is an assignment σ that satisfies all constraints, we can for each constraint φ, take the set corresponding to σ restricted on the variables involved in φ. By looking into each variable gadget, it's easy to see those sets are non-intersecting. Thus we can take m non-intersecting sets out of the $\eta \cdot m$ sets.

Soundness. Suppose any assignment satisfies at most μ fraction of constraints. Then we prove one can take at most $\mu \cdot m$ many ε-almost non-intersecting sets, for $\varepsilon < \frac{\gamma}{2q}$ where γ is the fraction of intersection of two hyperedges from two different matchings in the gadget.

Note that in order to be ε-almost non-intersecting, each set can drop at most an ε fraction of elements. However, any two sets that correspond to inconsistent assignments share $\frac{\gamma}{q} > 2\varepsilon$ fraction of elements. Therefore, the only way to pick ε-almost non-intersecting sets is to fix some global assignment σ of the CSP instance, and pick sets that correspond to constraints satisfied by σ. Given σ satisfies at most μ fraction of constraints, we can take at most $\mu \cdot m$ ε-almost non-intersecting sets in total.

Construction of the gadget. We use Feige's hypercube partition system [Fei98] as the gadget. Specifically, the gadget $G^{(x)} = (V^{(x)}, E^{(x)})$ is constructed as follows:

- $V^{(x)} = [d]^{\Sigma}$. We can think of $V^{(x)}$ as $|\Sigma|$-long vectors, where each entry takes a value in $[d]$.
- $e^{(x)}_{i,j} = \{a \in [d]^{\Sigma} \mid a_i = j\}$, i.e., vectors whose i-th entry is j.

It is easy to see each $E^{(x)}_i = \{e^{(x)}_{i,j}\}_{j\in[d]}$ is a matching. Furthermore, $|e^{(x)}_{i,j}| = d^{|\Sigma|-1}$, and for $i_1 \neq i_2$ and $j_1, j_2 \in [d]$, $|e^{(x)}_{i_1,j_1} \cap e^{(x)}_{i_2,j_2}| = d^{|\Sigma|-2}$. In other words, two hyperedges from two different matchings intersect in $\frac{1}{d}$ of their sizes.

Choice of Parameters. We use Feige's degree-5 3-SAT hardness [Fei98] as a starting point.

Lemma 8 ([Fei98]). *There is a universal constant $\varepsilon_0 > 0$ such that it is NP-hard to distinguish between the case where a 3-CNF formula φ is fully satisfiable, and the case where any assignment can satisfy at most $1 - \varepsilon_0$ fraction of clauses. Furthermore, this holds even when each clause has length exactly 3 and each variable appears exactly 5 times.*

Plugging everything into the reduction, we have:

- $q = 3$.
- $|\Sigma| = 2$.
- $d = 5$.
- $\eta = 7$, since each clause has exactly 7 satisfying assignments.
- $\gamma = \frac{1}{d} = \frac{1}{5}$.
- $\varepsilon < \frac{\gamma}{2q} = \frac{1}{30}$.
- $\alpha = \frac{1}{\eta} = \frac{1}{7}$.
- $\beta = \frac{1-\varepsilon_0}{\eta} = \frac{1}{7}(1 - \varepsilon_0)$.
- $|\mathcal{S}| = \eta \cdot m = 7 \cdot m$.
- Every $S_i \in \mathcal{S}$ has size $k = q \cdot d^{|\Sigma|-1} = 15$.
- $|U| = k \cdot m = 15 \cdot m$.

This finishes the proof of Theorem 4.

6 Conclusions

In this paper, we studied the bicriteria versions of several important scheduling problems, namely Makespan, Santa Claus, and Set Packing.

For Makespan, we provided a bicriteria approximation algorithm: we can schedule more than 0.6533 fraction of the jobs within makespan $(3/2) \cdot T$, where T is the optimal makespan to schedule all jobs. We suggest as future work to prove hardness or algorithmic results for scheduling a $(1 - 1/e + \varepsilon)$ fraction of jobs in makespan $(3/2 - \delta) \cdot T$, for $\varepsilon > 0.02$ and δ being a small constant. Another trade-off point that would be interesting future work is scheduling a $(1 - \varepsilon)$ fraction of jobs in makespan $(2 - \delta) \cdot T$, for ε, δ small constants.

For Santa Claus, we provided hardness results with a constant fraction of agent rejections. An interesting open question would be closing the ε vs. $(1 - \varepsilon)$ gap between the algorithm in [Gol05] and the hardness from us on the value each agent receives.

As a by-product, for Set Packing, we provided the hardness for $(1 - \delta_0, 1 - \varepsilon)$-bicriteria for some $\delta_0, \varepsilon > 0$, and gave an algorithm for $(1 - \delta, \varepsilon)$-bicriteria where δ can be arbitrarily close to 0. We conjecture $\varepsilon = 1/2$ might be the threshold of going from easy to hard, since when $\varepsilon < 1/2$, it's possible for a pair of sets to both pick an ε-fraction so that the subsets are disjoint, even if the two sets are identical. Thus, we suggest as future work to study the case when ε is around $1/2$.

References

[ABK01] Eric Angel, Evripidis Bampis, and Alexander Kononov. A fptas for approximating the unrelated parallel machines scheduling problem with costs. In *European Symposium on Algorithms*, pages 194–205. Springer, 2001.

[AKS17] Chidambaram Annamalai, Christos Kalaitzis, and Ola Svensson. Combinatorial algorithm for restricted max-min fair allocation. *ACM Transactions on Algorithms (TALG)*, 13(3):1–28, 2017.

[AS07] Arash Asadpour and Amin Saberi. An approximation algorithm for max-min fair allocation of indivisible goods. In *Proceedings of the thirty-ninth annual ACM symposium on Theory of computing*, pages 114–121, 2007.

[Bam25] Etienne Bamas. Lift-and-project integrality gaps for santa claus. *SODA*, 2025.

[BCG09] MohammadHossein Bateni, Moses Charikar, and Venkatesan Guruswami. Maxmin allocation via degree lower-bounded arborescences. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 543–552, 2009.

[BD05] Ivona Bezáková and Varsha Dani. Allocating indivisible goods. *ACM SIGecom Exchanges*, 5(3):11–18, 2005.

[Ber00] Piotr Berman. A d/2 approximation for maximum weight independent set in d-claw free graphs. *Nord. J. Comput.*, 7(3):178–184, 2000.

[BLM+24] Étienne Bamas, Alexander Lindermayr, Nicole Megow, Lars Rohwedder, and Jens Schlöter. Santa claus meets makespan and matroids: Algorithms and reductions. In David P. Woodruff, editor, *Proceedings of the 2024 ACM-SIAM Symposium on Discrete Algorithms, SODA 2024, Alexandria, VA, USA, January 7-10, 2024*, pages 2829–2860. SIAM, 2024.

[BLMS+00] Yair Bartal, Stefano Leonardi, Alberto Marchetti-Spaccamela, Jiri Sgall, and Leen Stougie. Multiprocessor scheduling with rejection. *SIAM Journal on Discrete Mathematics*, 13(1):64–78, 2000.

[BR23] Étienne Bamas and Lars Rohwedder. Better trees for santa claus. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, pages 1862–1875, 2023.

[BS06] Nikhil Bansal and Maxim Sviridenko. The santa claus problem. In *Proceedings of the thirty-eighth annual ACM symposium on Theory of computing*, pages 31–40, 2006.

[CCK09] Deeparnab Chakrabarty, Julia Chuzhoy, and Sanjeev Khanna. On allocating goods to maximize fairness. In *50th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2009, October 25-27, 2009, Atlanta, Georgia, USA*, pages 107–116. IEEE Computer Society, 2009.

[CG10] Deeparnab Chakrabarty and Gagan Goel. On the approximability of budgeted allocations and improved lower bounds for submodular welfare maximization and GAP. *SIAM J. Comput.*, 39(6):2189–2211, 2010.

[CK05] Chandra Chekuri and Sanjeev Khanna. A polynomial time approximation scheme for the multiple knapsack problem. *SIAM Journal on Computing*, 35(3):713–728, 2005.

[CM18] Siu-Wing Cheng and Yuchen Mao. Restricted max-min fair allocation. *arXiv preprint arXiv:1804.10902*, 2018.

[CM19] Siu-Wing Cheng and Yuchen Mao. Restricted max-min allocation: Approximation and integrality gap. *arXiv preprint arXiv:1905.06084*, 2019.

[Cyg13] Marek Cygan. Improved approximation for 3-dimensional matching via bounded pathwidth local search. In *54th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2013, Berkeley, CA, USA, October, 26-29, 2013*, pages 509–518. IEEE Computer Society, 2013.

[DRZ20] Sami Davies, Thomas Rothvoss, and Yihao Zhang. A tale of santa claus, hypergraphs and matroids. In *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 2748–2757. SIAM, 2020.

[EKS14] Tomás Ebenlendr, Marek Krcál, and Jirí Sgall. Graph balancing: A special case of scheduling unrelated parallel machines. *Algorithmica*, 68(1):62–80, 2014.

[ES99] Leah Epstein and Jiří Sgall. Approximation schemes for scheduling on uniformly related and identical parallel machines. In *Algorithms-ESA'99: 7th Annual European Symposium Prague, Czech Republic, July 16–18, 1999 Proceedings 7*, pages 151–162. Springer, 1999.

[Fei98] Uriel Feige. A threshold of ln n for approximating set cover. *J. ACM*, 45(4):634–652, 1998.

[Fei04] Uriel Feige. Approximating maximum clique by removing subgraphs. *SIAM J. Discret. Math.*, 18(2):219–225, 2004.

[Fei08] Uriel Feige. On allocations that maximize fairness. In *SODA*, volume 8, pages 287–293, 2008.

[FGMS06] Lisa Fleischer, Michel X. Goemans, Vahab S. Mirrokni, and Maxim Sviridenko. Tight approximation algorithms for maximum general assignment problems. In *Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2006, Miami, Florida, USA, January 22-26, 2006*, pages 611–620. ACM Press, 2006.

[FV06] Uriel Feige and Jan Vondrák. Approximation algorithms for allocation problems: Improving the factor of 1 - 1/e. In *47th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2006, Berkeley, California, USA, October 21-24, 2006, Proceedings*, pages 667–676. IEEE Computer Society, 2006.

[Gol05] Daniel Golovin. *Max-min fair allocation of indivisible goods.* School of Computer Science, Carnegie Mellon University, 2005.

[Gra69] Ronald L. Graham. Bounds on multiprocessing timing anomalies. *SIAM journal on Applied Mathematics*, 17(2):416–429, 1969.

[GWL+23] Jatinder ND Gupta, Chin-Chia Wu, Win-Chin Lin, Xin-Gong Zhang, Danyu Bai, Bertrand MT Lin, and Chia-Cheng Liao. Bicriteria single-machine scheduling with multiple job classes and customer orders. *Applied Soft Computing*, 147:110809, 2023.

[Hal95] Magnús M. Halldórsson. Approximating discrete collections via local improvements. In Kenneth L. Clarkson, editor, *Proceedings of the Sixth Annual ACM-SIAM Symposium on Discrete Algorithms, 22-24 January 1995. San Francisco, California, USA*, pages 160–169. ACM/SIAM, 1995.

[Hås99] Johan Håstad. Clique is hard to approximate within n 1- ε. 1999.

[HKT00] Magnús M Halldórsson, Jan Kratochvıl, and Jan Arne Telle. Independent sets with domination constraints. *Discrete Applied Mathematics*, 99(1-3):39–54, 2000.

[HO15] Chien-Chung Huang and Sebastian Ott. A combinatorial approximation algorithm for graph balancing with light hyper edges. *arXiv preprint arXiv:1507.07396*, 2015.

[HS76] Ellis Horowitz and Sartaj Sahni. Exact and approximate algorithms for scheduling nonidentical processors. *Journal of the ACM (JACM)*, 23(2):317–327, 1976.

[HS89] Cor A. J. Hurkens and Alexander Schrijver. On the size of systems of sets every t of which have an sdr, with an application to the worst-case ratio of heuristics for packing problems. *SIAM J. Discret. Math.*, 2(1):68–72, 1989.

[HS23] Penny Haxell and Tibor Szabó. Improved integrality gap in max-min allocation: or topology at the north pole. In *Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2875–2897. SIAM, 2023.

[HSS06] Elad Hazan, Shmuel Safra, and Oded Schwartz. On the complexity of approximating k-set packing. *Comput. Complex.*, 15(1):20–39, 2006.

[HSW03] Han Hoogeveen, Martin Skutella, and Gerhard J Woeginger. Preemptive scheduling with rejection. *Mathematical Programming*, 94:361–374, 2003.

[JP99] Klaus Jansen and Lorant Porkolab. Improved approximation schemes for scheduling unrelated parallel machines. In *Proceedings of the thirty-first annual ACM Symposium on Theory of Computing*, pages 408–417, 1999.

[JR17] Klaus Jansen and Lars Rohwedder. On the configuration-lp of the restricted assignment problem. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 2670–2678. SIAM, 2017.

[JR19] Klaus Jansen and Lars Rohwedder. Local search breaks 1.75 for graph balancing. In Christel Baier, Ioannis Chatzigiannakis, Paola Flocchini, and Stefano Leonardi, editors, *46th International Colloquium on Automata, Languages, and Programming, ICALP 2019, July 9-12, 2019, Patras, Greece*, volume 132 of *LIPIcs*, pages 74:1–74:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019.

[LG23] Shuguang Li and Zhichao Geng. Bicriteria scheduling on an unbounded parallel-batch machine for minimizing makespan and maximum cost. *Information Processing Letters*, 180:106343, 2023.

[LL20] Xiaofei Liu and Weidong Li. Approximation algorithm for the single machine scheduling problem with release dates and submodular rejection penalty. *Mathematics*, 8(1):133, 2020.

[LL21] Xiaofei Liu and Weidong Li. Approximation algorithms for the multiprocessor scheduling with submodular penalties. *Optimization Letters*, 15:2165–2180, 2021.

[LOSZ20] André Linhares, Neil Olver, Chaitanya Swamy, and Rico Zenklusen. Approximate multi-matroid intersection via iterative refinement. *Math. Program.*, 183(1):397–418, 2020.

[LST90] Jan Karel Lenstra, David B. Shmoys, and Éva Tardos. Approximation algorithms for scheduling unrelated parallel machines. *Math. Program.*, 46:259–271, 1990.

[LST24] Euiwoong Lee, Ola Svensson, and Theophile Thiery. Asymptotically optimal hardness for k-set packing and k-matroid intersection, 2024.

[LSV13] Jon Lee, Maxim Sviridenko, and Jan Vondrák. Matroid matching: The power of local search. *SIAM J. Comput.*, 42(1):357–379, 2013.

[MW15] Matthias Mnich and Andreas Wiese. Scheduling and fixed-parameter tractability. *Mathematical Programming*, 154(1):533–562, 2015.

[NBY06] Zeev Nutov, Israel Beniaminy, and Raphael Yuster. A (1-1/e)-approximation algorithm for the generalized assignment problem. *Oper. Res. Lett.*, 34(3):283–288, 2006.

[Neu21] Meike Neuwohner. An improved approximation algorithm for the maximum weight independent set problem in d-claw free graphs. In Markus Bläser and Benjamin Monmege, editors, *38th International Symposium on Theoretical Aspects of Computer Science, STACS 2021, March 16-19, 2021, Saarbrücken, Germany (Virtual Conference)*, volume 187 of *LIPIcs*, pages 53:1–53:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.

[Neu23] Meike Neuwohner. Passing the limits of pure local search for weighted k-set packing. In Nikhil Bansal and Viswanath Nagarajan, editors, *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023*, pages 1090–1137. SIAM, 2023.

[PWW20] Zhi Pei, Mingzhong Wan, and Ziteng Wang. A new approximation algorithm for unrelated parallel machine scheduling with release dates. *Annals of Operations Research*, 285(1):397–425, 2020.

[PY00] Christos H Papadimitriou and Mihalis Yannakakis. On the approximability of trade-offs and optimal access of web sources. In *Proceedings 41st annual symposium on foundations of computer science*, pages 86–92. IEEE, 2000.

[SG12] Dvir Shabtay and Nufar Gasper. Two-machine flow-shop scheduling with rejection. *Computers & Operations Research*, 39(5):1087–1096, 2012.

[SGK13] Dvir Shabtay, Nufar Gaspar, and Moshe Kaspi. A survey on offline scheduling with rejection. *Journal of scheduling*, 16:3–28, 2013.

[Sve11] Ola Svensson. Santa claus schedules jobs on unrelated machines. In *Proceedings of the forty-third annual ACM symposium on Theory of computing*, pages 617–626, 2011.

[TB06] Vincent T'kindt and Jean-Charles Billaut. *Multicriteria scheduling: theory, models and algorithms*. Springer Science & Business Media, 2006.

[VW14] José Verschae and Andreas Wiese. On the configuration-lp for scheduling on unrelated machines. *Journal of Scheduling*, 17:371–383, 2014.

[Woe97] Gerhard J Woeginger. A polynomial-time approximation scheme for maximizing the minimum machine completion time. *Operations Research Letters*, 20(4):149–154, 1997.

[Zuc06] David Zuckerman. Linear degree extractors and the inapproximability of max clique and chromatic number. In *Proceedings of the thirty-eighth annual ACM symposium on Theory of computing*, pages 681–690, 2006.